Try Except

1.Try

The try keyword is used to test a block of code for errors (exceptions) without stopping the entire program.

2.except

The except keyword is used to handle errors (exceptions) that occur inside the try block.

In [10]:
```
try:
    print(x)
except:
    print("error")
```

```
error
```

In [12]:
```
try:
    print(x)
except NameError:
    print("x is not defined")
except:
    print("error")
```

```
x is not defined
```

In [14]:
```
try:
    a=10
    b=0
    print(10/0)
except ZeroDivisionError:
    print("is not possible 0 not divide with number")
except NameError:
    print("variable is not defind")
except:
    print("Nothing else")
```

```
is not possible 0 not divide with number
```

3.else

The else keyword is used with try and except. It executes only when the try block completes successfully without any error.

In [17]:
```
try:
    print("hello")
except:
    print("nothing")
else:
    print("every thing is right")
```

```
hello
every thing is right
```

In [20]:
```
try:
    a=10
    b=0
    print(a/b)
except:
    print("not possible")
else:
    print("try successfully execute")
```

```
not possible
```

Exception

An exception is an error that occurs during the execution of a Python program and

interrupts its normal flow.

In [9]:
```
try:
    a=10
    b=int("hi")
    print(a/b)
except Exception as e:
    print(e)
```

```
invalid literal for int() with base 10: 'hi'
```

In [23]:
```
try:
    a=10
    b=0
    print(a/b)
except Exception as e:
    print(e)
```

```
division by zero
```

4.finally

The finally keyword is used to execute a block of code whether an exception occurs or not.

In [28]:
```
try:
    a = 10
    b = 2
    print(a / b)

except ZeroDivisionError:
    print("Cannot divide by zero")

else:
    print("No error occurred")

finally:
    print("Program completed")
```

```
5.0
No error occurred
Program completed
```

In [30]:
```
try:
    print(10/0)
except:
    print("A")
else:
    print("B")
finally:
    print("C")
```

```
A
C
```

5.Raise

The raise keyword is used to manually generate an exception in Python when a specific condition is not satisfied.

In [36]:
```
age = 10

if age < 18:
    raise ValueError("Age must be 18 or above")

print("Age is valid")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[36], line 4
      1 age = 10
      3 if age < 18:
----> 4     raise ValueError("Age must be 18 or above")
      6 print("Age is valid")

ValueError: Age must be 18 or above
```

Practice Question

1. Division Calculator

Write a program to divide two numbers using try and except.

- If the second number is zero, handle ZeroDivisionError.

- Otherwise, print the result.

In [3]:
```
a=10
b=0
try:
    print(a/b)
except ZeroDivisionError:
    print("0 is not divisible with any number")
```

```
0 is not divisible with any number
```

In [5]:
```
a=12
b=2
try:
    print(a/b)
except ZeroDivisionError:
    print("0 is not divisible with any number")
```

```
6.0
```

2. Valid Integer Input

Write a program to get an integer from the user.

- If the user enters a string instead of a number, handle ValueError.

- Otherwise, print the number.

In [16]:
```
try:
    a=int(input("enter the num: "))
    print(a)
except ValueError:
    print("Please enter a valid integer")
```

```
Please enter a valid integer
```

3. Multiple Exceptions

Write a program to divide two user-input numbers.

- Handle ValueError.

- Handle ZeroDivisionError.

- Use separate except blocks.

In [25]:
```
try:
    a=int(input("enter a:"))
    b=int(input("enter b:"))
    print(a/b)
except ValueError:
    print("this is invalied")
except ZeroDivisionError:
    print("0 is not divisible")
```

```
2.0
```

In [27]:
```
try:
    a=int(input("enter a:"))
```

```
    b=int(input("enter b:"))
    print(a/b)
except ValueError:
    print("this is invalied")
except ZeroDivisionError:
    print("0 is not divisible")
```

```
this is invalied
```

In [29]:
```
try:
    a=int(input("enter a:"))
    b=int(input("enter b:"))
    print(a/b)
except ValueError:
    print("this is invalied")
except ZeroDivisionError:
    print("0 is not divisible")
```

```
0 is not divisible
```

4. Use of else

Write a program to convert a string into an integer.

- Use try to convert.

- Use except to handle invalid input.

- Use else to print "Conversion successful".

In [38]:
```
a="100"
try:
    b=int(a)
    print(b)
except:
    print("invalied input")
else:
    print("conversion successful")
```

```
100
conversion successful
```

In [40]:
```
a="Reeta"
try:
    b=int(a)
    print(b)
except:
    print("invalied input")
else:
    print("conversion successful")
```

```
invalied input
```

5. Use of finally

Write a program to divide two numbers.

- Handle division by zero.

- Use finally to print "Program completed".

In [45]:
```
a=10
b=0
try:
    print(a/b)
except ZeroDivisionError:
    print("0 is not divisible")
finally:
    print("Program completed")
```

```
0 is not divisible
Program completed
```

In [47]:
```
a=10
```

```
b=5
try:
    print(a/b)
except ZeroDivisionError:
    print("0 is not divisible")
finally:
    print("Program completed")
```

```
2.0
Program completed
```

6. Age Validation using raise

Write a program to check voting eligibility.

- If age is below 18, raise ValueError.

- Otherwise, print "Eligible for voting".

- Handle the exception using except.

In [54]:
```
age=int(input("enter num:"))
try:
    if age < 18:
        raise ValueError("must age is above 18")
    print("Eligible for voting")
except Exception as e:
    print(e)
```

```
must age is above 18
```

7. Positive Number Validation

Write a program to get a number from the user.

- If the number is negative, raise ValueError.

- Otherwise, print "Valid positive number".

In [61]:
```
try:
    a = int(input("Enter num: "))
    if a < 0:
        raise ValueError("Negative number")
    print("Valid positive number")
except ValueError as e:
    print(e)
```

```
Valid positive number
```

8. Bank Withdrawal System

Write a program to withdraw money.

- Get the available balance.

- Get the withdrawal amount.

- If the withdrawal amount is greater than the balance, raise ValueError.

- Otherwise, print "Withdrawal successful".

In [68]:
```
balance=1000
withdrawal=500
try:
```

```
    if withdrawal > balance:
        raise ValueError("not avaliable")
    print("Withdrawal Successful")
except Exception as e:
    print(e)
```

```
Withdrawal Successful
```

9.Student Mark Validation

Write a program to validate student marks.

- Get marks from the user.

- If marks are below 0 or above 100, raise ValueError.

- Otherwise, print "Valid marks".

- Use try, except, else, and finally.

In [80]:
```
mark=150
try:
    if mark <0 or mark >100:
        raise ValueError("not valied")
    print("Valid marks")
except Exception as e:
    print(e)
else:
    print("present")
finally:
    print("student attend the test")
```

```
not valied
student attend the test
```

Loading [MathJax]/extensions/Safe.js